

```
>>> dq = deque([1,2,3])
>>> dq.pop()
3
>>> dq = deque([1,2,3])
>>> dq.popleft()
1
```

- In order to find an element and retrieve its index, we need not iterate through the entire deque, rather we can use the `index()` method which takes three parameters, of which two are optional. It takes in the element, the `start index` and the `end index` as parameters of which the latter two are optional. It returns the position of the element at or after the start index and before the `end index`. It should be noted that element at the `start index` is included in the search but the element at `end index` is not.

The method signature is `index(element, start, end)` and all indexes are zero-based indexes.

```
>>> dq = deque([5, 4, 3, 2, 1])
>>> dq.index(5, 0, 2)           #search 5 in indexes
                                #from 0 to 1
0                               #index of element (5)
>>> dq.index(4, 2, 5)           #search 4 in indexes
                                #from 2 to 4
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: 4 is not in deque
```

Even though 4 is present in the `deque` object we get an error because we supplied the start index as 2 and end index as 5, which forces the search only in the slice of deque (3, 2, 1), hence the search fails and we encounter a `ValueError` **d**